La Trattoria

API REST

Sistema de gestion de pedidos para restaurante

Spring Boot · MySQL · JWT · QR Code · REST

https://api.latrattoria.com/api/v1

GET	/api/v1/platos	Listar todos los platos activos
POST	/api/v1/pedidos	Crear nuevo pedido desde QR
GET	/api/v1/mesas/qr/{token}	Validar mesa por codigo QR
PATCH	/api/v1/pedidos/{id}/estado	Actualizar estado del pedido
POST	/api/v1/auth/login	Login admin y personal
GET	/api/v1/categorias	Listar categorias de la carta
DELETE	/api/v1/pedidos/{id}	Cancelar pedido

La Trattoria · Documentacion API REST

1. Introducción

API REST para un restaurante con sistema de pedidos por QR de mesa. Los clientes acceden a la carta escaneando el QR de su mesa y realizan pedidos de forma completamente anónima. El personal y administradores acceden mediante login con credenciales y reciben un token JWT.

Base URL	`https://api.latrattoria.com/api/v1`
Protocolo	`HTTPS`
Formato	`JSON (application/json)`
Autenticación	`JWT Bearer Token (solo admin/personal)`
Framework	`Spring Boot 3.x · Java 17`
Base de datos	`MySQL 8`

Flujos principales

Flujo cliente (anónimo): El cliente escanea el QR de la mesa → la URL contiene el token de mesa → accede a la carta → añade platos → confirma el pedido, que queda vinculado automáticamente a la mesa.

Flujo personal/admin: Accede por URL especial → login con email y password → recibe JWT → gestiona pedidos, carta, mesas y usuarios.

2. Autenticación

Solo los endpoints de gestión (admin y personal) requieren autenticación. Los endpoints de carta y pedidos son públicos (acceso anónimo desde el QR).

2.1 Login

POST	`/api/v1/auth/login`	Obtener token JWT con credenciales — *Público*

Body (JSON):

Campo	Tipo	Req.	Descripción
`email`	String	Sí	Email del usuario del sistema
`password`	String	Sí	Contraseña en texto plano (se compara con bcrypt)

Ejemplo de petición:

```
{ "email": "admin@restaurante.com", "password": "Admin1234!" }
```

Respuesta 200 OK:

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "rol": "administrador", "nombre":
"Admin Principal" }
```

Códigos de respuesta:

Código	Estado HTTP	Descripción
`200`	**OK**	Login correcto. Devuelve JWT.
`400`	**Bad Request**	Faltan campos obligatorios.
`401`	**Unauthorized**	Credenciales incorrectas.
`403`	**Forbidden**	Usuario inactivo o sin permisos.

2.2 Uso del token

Todos los endpoints protegidos requieren el siguiente header:

```
Authorization: Bearer {token}
```

El token tiene validez de 8 horas. Si expira, el servidor devuelve 401.

3. Gestión de Errores Globales

La API devuelve siempre una respuesta JSON estructurada ante cualquier error, gestionada mediante un @ControllerAdvice global en Spring Boot. Nunca se expone información interna del servidor.

Formato de error estándar:

```
{ "timestamp": "2025-02-20T13:00:00", "status": 404, "error": "Not Found", "mensaje":
"Plato no encontrado con id: 99" }
```

Tabla de códigos HTTP utilizados en la API:

Código	Estado HTTP	Descripción
200	**OK**	Petición correcta. Devuelve el recurso solicitado.
201	**Created**	Recurso creado correctamente (POST).
204	**No Content**	Operación correcta sin cuerpo de respuesta (DELETE).
400	**Bad Request**	Datos de entrada inválidos o falta un campo obligatorio.
401	**Unauthorized**	Token JWT ausente, inválido o expirado.
403	**Forbidden**	Sin permisos para realizar esta operación.
404	**Not Found**	El recurso solicitado no existe.
409	**Conflict**	Conflicto: email duplicado, mesa ocupada, etc.
422	**Unprocessable Entity**	Datos con formato correcto pero semánticamente inválidos.
500	**Internal Server Error**	Error inesperado del servidor.

4. Módulo: Carta (Categorías y Platos)

Endpoints públicos para que el cliente vea la carta, y endpoints protegidos para que admin/personal gestione categorías y platos.

4.1 Categorías

GET	`/api/v1/categorias`	Listar todas las categorías — *Público*
GET	`/api/v1/categorias/{id}`	Obtener categoría por ID — *Público*
POST	`/api/v1/categorias`	Crear nueva categoría — *Admin*
PUT	`/api/v1/categorias/{id}`	Actualizar categoría — *Admin*
DELETE	`/api/v1/categorias/{id}`	Eliminar categoría — *Admin*

Códigos de respuesta:

Código	Estado HTTP	Descripción
200	**OK**	Lista o categoría obtenida correctamente.
201	**Created**	Categoría creada correctamente.
204	**No Content**	Categoría eliminada.
400	**Bad Request**	Nombre de categoría inválido.
401	**Unauthorized**	Token ausente o inválido.
403	**Forbidden**	Solo admin puede gestionar categorías.
404	**Not Found**	Categoría no encontrada.
409	**Conflict**	Ya existe una categoría con ese nombre.

4.2 Platos

GET	`/api/v1/platos`	Listar todos los platos activos — *Público*
GET	`/api/v1/platos/{id}`	Obtener plato por ID — *Público*
GET	`/api/v1/platos/categoria/{idCat}`	Listar platos por categoría — *Público*
POST	`/api/v1/platos`	Crear nuevo plato — *Admin*

PUT	`/api/v1/platos/{id}`	Actualizar plato — *Admin*

PATCH	`/api/v1/platos/{id}/disponibilidad`	Cambiar disponibilidad (activo/inactivo) — *Admin/Personal*

DELETE	`/api/v1/platos/{id}`	Eliminar plato — *Admin*

Campos del body para crear/actualizar plato:

Campo	Tipo	Req.	Descripción
`nombre`	String	Sí	Nombre del plato
`descripcion`	String	No	Descripción del plato
`precio`	Decimal	Sí	Precio en euros (ej: 12.50)
`imagen`	String	No	Nombre del archivo de imagen
`disponibilidad`	String	No	activo | inactivo (default: activo)
`idCategoria`	Integer	Sí	ID de la categoría

Códigos de respuesta:

Código	Estado HTTP	Descripción
`200`	**OK**	Plato/s obtenido/s correctamente.
`201`	**Created**	Plato creado correctamente.
`204`	**No Content**	Plato eliminado.
`400`	**Bad Request**	Datos inválidos (precio negativo, campos vacíos...).
`401`	**Unauthorized**	Token ausente o inválido.
`403`	**Forbidden**	Sin permisos para esta operación.
`404`	**Not Found**	Plato o categoría no encontrados.
`422`	**Unprocessable Entity**	Precio con formato incorrecto.

5. Módulo: Mesas y QR

Las mesas tienen un código QR único. Cuando el cliente escanea el QR, accede a la URL /api/v1/mesas/qr/{codigoQr} que valida la mesa y devuelve su información para iniciar el flujo de pedido.

GET	`/api/v1/mesas/qr/{codigoQr}`	Validar QR y obtener info de la mesa — *Público*
GET	`/api/v1/mesas`	Listar todas las mesas — *Admin/Personal*
GET	`/api/v1/mesas/{id}`	Obtener mesa por ID — *Admin/Personal*
POST	`/api/v1/mesas`	Crear nueva mesa — *Admin*
PUT	`/api/v1/mesas/{id}`	Actualizar mesa — *Admin*
PATCH	`/api/v1/mesas/{id}/disponible`	Cambiar disponibilidad de la mesa — *Admin/Personal*
DELETE	`/api/v1/mesas/{id}`	Eliminar mesa — *Admin*

Respuesta de GET /api/v1/mesas/qr/{codigoQr} (200 OK):

```
{ "idMesa": 3, "numero": 3, "capacidad": 4, "disponible": true, "codigoQr":
"qr-mesa-i9j0kll2-0003" }
```

Códigos de respuesta:

Código	Estado HTTP	Descripción
200	**OK**	Mesa obtenida o lista de mesas correcta.
201	**Created**	Mesa creada correctamente.
204	**No Content**	Mesa eliminada.
400	**Bad Request**	Datos inválidos al crear/actualizar mesa.
401	**Unauthorized**	Token ausente o inválido.
403	**Forbidden**	Sin permisos (solo admin/personal).
404	**Not Found**	Mesa o código QR no encontrados.
409	**Conflict**	Número de mesa o código QR ya existente.

6. Módulo: Pedidos (flujo cliente anónimo)

El cliente no necesita registrarse. El pedido se vincula a la mesa mediante el codigoQr obtenido al escanear. El estado del pedido sigue el ciclo: nuevo → en preparación → listo → entregado.

POST	`/api/v1/pedidos`	Crear nuevo pedido (anónimo desde QR) — *Público*
GET	`/api/v1/pedidos/{id}`	Consultar estado de un pedido — *Público*
GET	`/api/v1/pedidos`	Listar todos los pedidos — *Admin/Personal*
GET	`/api/v1/pedidos/mesa/{idMesa}`	Listar pedidos de una mesa — *Admin/Personal*
GET	`/api/v1/pedidos/estado/{estado}`	Filtrar pedidos por estado — *Admin/Personal*
PATCH	`/api/v1/pedidos/{id}/estado`	Actualizar estado del pedido — *Admin/Personal*
DELETE	`/api/v1/pedidos/{id}`	Cancelar/eliminar pedido — *Admin*

Body para crear pedido (POST /api/v1/pedidos):

Campo	Tipo	Req.	Descripción
`codigoQr`	String	Sí	Token QR de la mesa (obtenido al escanear)
`modalidad`	String	Sí	mesa | recogida
`metodoPago`	String	Sí	tarjeta | PayPal | Bizum | efectivo
`detalles`	Array	Sí	Lista de platos pedidos (ver módulo 7)

Body para actualizar estado (PATCH /api/v1/pedidos/{id}/estado):

```
{ "estado": "en preparación" }   // Valores: nuevo | en preparación | listo | entregado | cancelado
```

Códigos de respuesta:

Código	Estado HTTP	Descripción
200	**OK**	Pedido/s obtenido/s o estado actualizado.
201	**Created**	Pedido creado correctamente.

204	**No Content**	Pedido eliminado.
400	**Bad Request**	Datos inválidos, mesa no disponible o QR incorrecto.
401	**Unauthorized**	Token ausente o inválido.
403	**Forbidden**	Sin permisos para esta operación.
404	**Not Found**	Pedido o mesa no encontrados.
409	**Conflict**	La mesa ya tiene un pedido activo.
422	**Unprocessable Entity**	Estado de pedido no permitido en la transición actual.

7. Módulo: Detalle de Pedido

Cada pedido contiene una o más líneas de detalle. Normalmente se crean junto con el pedido (incluidos en el array detalles), pero también se pueden añadir líneas a un pedido existente mientras esté en estado nuevo.

GET	`/api/v1/pedidos/{idPedido}/detalles`	Listar detalles de un pedido — *Público*
POST	`/api/v1/pedidos/{idPedido}/detalles`	Añadir línea a un pedido nuevo — *Público*
PUT	`/api/v1/pedidos/{idPedido}/detalles/{id}`	Modificar línea de detalle — *Admin/Personal*
DELETE	`/api/v1/pedidos/{idPedido}/detalles/{id}`	Eliminar línea de detalle — *Admin/Personal*

Campos de cada línea de detalle:

Campo	Tipo	Req.	Descripción
`idPlato`	Integer	Sí	ID del plato pedido
`cantidad`	Integer	Sí	Cantidad (mínimo 1)
`instruccionesEspeciales`	String	No	Alergias, preferencias, etc.

Códigos de respuesta:

Código	Estado HTTP	Descripción
200	**OK**	Detalle/s obtenido/s o modificado correctamente.
201	**Created**	Línea de detalle añadida.
204	**No Content**	Línea eliminada.
400	**Bad Request**	Cantidad <= 0 o plato no disponible.
404	**Not Found**	Pedido, detalle o plato no encontrados.
409	**Conflict**	No se puede modificar un pedido que ya no está en estado nuevo.

8. Módulo: Notificaciones

Las notificaciones se vinculan al pedido (no al cliente, ya que es anónimo). Permiten informar del estado del pedido en tiempo real, y al personal de promociones o novedades desde la pantalla de gestión.

GET	`/api/v1/notificaciones/pedido/{idPedido}`	Ver notificaciones de un pedido — *Público*
GET	`/api/v1/notificaciones`	Listar todas las notificaciones — *Admin/Personal*
GET	`/api/v1/notificaciones/no-leidas`	Listar notificaciones no leídas — *Admin/Personal*
POST	`/api/v1/notificaciones`	Crear notificación (promo o novedad) — *Admin*
PATCH	`/api/v1/notificaciones/{id}/leida`	Marcar notificación como leída — *Admin/Personal*
DELETE	`/api/v1/notificaciones/{id}`	Eliminar notificación — *Admin*

Campos para crear notificación:

Campo	Tipo	Req.	Descripción
`idPedido`	Integer	Sí	ID del pedido al que va asociada
`mensaje`	String	Sí	Texto de la notificación
`tipo`	String	Sí	estado pedido | promoción | novedad

Códigos de respuesta:

Código	Estado HTTP	Descripción
200	**OK**	Notificación/es obtenida/s correctamente.
201	**Created**	Notificación creada.
204	**No Content**	Notificación eliminada.
400	**Bad Request**	Tipo de notificación inválido o campos vacíos.
401	**Unauthorized**	Token ausente o inválido.
403	**Forbidden**	Sin permisos (solo admin/personal).
404	**Not Found**	Notificación o pedido no encontrados.

9. Módulo: Usuarios del Sistema

Gestión de los usuarios internos del restaurante. Solo el administrador puede crear, modificar o eliminar usuarios. El personal solo puede consultar su propio perfil.

GET	`/api/v1/usuarios`	Listar todos los usuarios — *Admin*
GET	`/api/v1/usuarios/{id}`	Obtener usuario por ID — *Admin*
GET	`/api/v1/usuarios/perfil`	Ver perfil propio (usuario logado) — *Admin/Personal*
POST	`/api/v1/usuarios`	Crear nuevo usuario — *Admin*
PUT	`/api/v1/usuarios/{id}`	Actualizar usuario — *Admin*
PATCH	`/api/v1/usuarios/{id}/activo`	Activar/desactivar usuario — *Admin*
DELETE	`/api/v1/usuarios/{id}`	Eliminar usuario — *Admin*

Campos para crear/actualizar usuario:

Campo	Tipo	Req.	Descripción
`nombre`	String	Sí	Nombre completo del usuario
`email`	String	Sí	Email único (usado como username)
`password`	String	Sí*	Contraseña (solo en creación o cambio)
`rol`	String	Sí	personal | administrador
`activo`	Boolean	No	true | false (default: true)

Códigos de respuesta:

Código	Estado HTTP	Descripción
200	**OK**	Usuario/s obtenido/s o actualizado.
201	**Created**	Usuario creado correctamente.
204	**No Content**	Usuario eliminado.
400	**Bad Request**	Datos inválidos o contraseña no cumple requisitos.
401	**Unauthorized**	Token ausente o inválido.
403	**Forbidden**	Solo el administrador puede gestionar usuarios.
404	**Not Found**	Usuario no encontrado.

`409`	**Conflict**	El email ya está registrado.

10. Resumen Global de Códigos HTTP

Referencia rápida de todos los códigos HTTP utilizados en La Trattoria API.

Código	Estado HTTP	Descripción
200	**OK**	Éxito. Devuelve el recurso o la lista solicitada.
201	**Created**	Recurso creado. Se devuelve con su ID asignado.
204	**No Content**	Operación exitosa sin datos que devolver (DELETE, PATCH simple).
400	**Bad Request**	Error en los datos enviados: campos faltantes o formatos incorrectos.
401	**Unauthorized**	No autenticado. Token JWT ausente, inválido o expirado.
403	**Forbidden**	Autenticado pero sin permiso para esta operación.
404	**Not Found**	El recurso con el ID especificado no existe.
409	**Conflict**	Dato único duplicado (email, número de mesa, código QR).
422	**Unprocessable Entity**	Datos con formato válido pero lógicamente incorrectos.
500	**Internal Server Error**	Error inesperado del servidor. Se registra en logs internos.